



Riphah International University, Islamabad
Faculty of Computing

Compiler Construction
Semester Project Report

Submitted by:
Wajahat Ali Khan
Sap I'd: 55431
Section: BSCS 6-1

Submitted To:
Mr. Tajamul Shahzad

Date of Submission:
04 May 2026

Table of Contents

Introduction.....	3
Theoretical Background.....	3
System Architecture.....	4
Executive Summary.....	5
User Interface Design.....	5
Testing and Validation.....	6
Output.....	7
Conclusion.....	9
Reference.....	9

Introduction

The LR(0) Parser Generator and Visualizer is a Python program that automatically builds an LR(0) parser from any grammar you provide.

In compiler design, creating an LR(0) parser by hand is difficult, time-consuming, and easy to get wrong. This tool solves that problem by doing all the complex work automatically.

It serves two main purposes:

1. **Educational Tool** - Helps students learn how LR(0) parsers work through interactive visuals
2. **Backend Engine** - Processes grammars, calculates item sets, finds state transitions, detects conflicts, and displays the resulting **DFA** and **parsing table** in a clear, graphical way.

Theoretical Background

Context-Free Grammar (CFG): A 4-tuple $G = (V_N, V_T, P, S)$ where V_N are non-terminals, V_T are terminals, P are productions ($A \rightarrow \alpha$), and S is the start symbol.

LR(0) Item: A production with a dot ($\bullet$) indicating parsing progress. Example: For $E \rightarrow E + T$, items are $E \rightarrow \bullet E + T$, $E \rightarrow E \bullet + T$, $E \rightarrow E + \bullet T$, $E \rightarrow E + T \bullet$.

Closure Operation: Expands an item set by adding all items for non-terminals appearing after a dot.

Goto Operation: Moves the dot past a symbol X in all applicable items and applies closure.

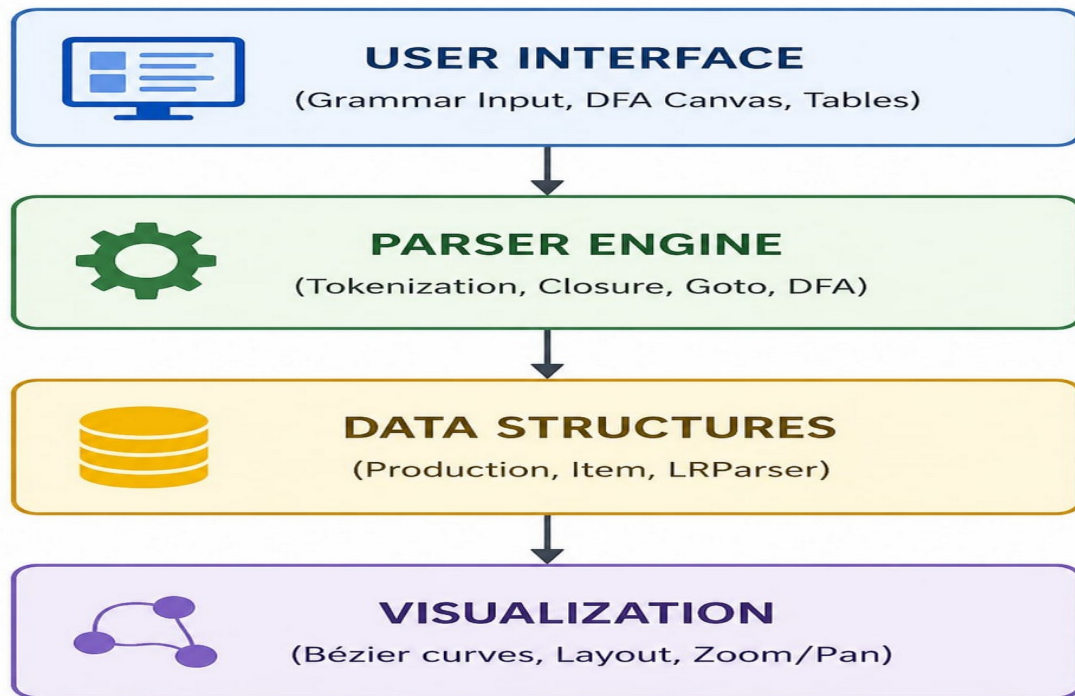
Canonical Collection: The set of all LR(0) states (item sets) reachable from the initial state via repeated GOTO operations.

Parsing Tables: ACTION table (for terminals) specifies shift/reduce/accept actions; GOTO table (for non-terminals) specifies state transitions.

Conflicts: A grammar is LR(0) if no state contains shift-reduce or reduce-reduce conflicts.

System Architecture

The system follows a modular layered architecture:



Module Responsibilities:

- **Tokenizer:** Converts RHS strings into symbol tokens
- **Production/Item:** Represent grammar rules and LR(0) items
- **LR0Parser:** Core algorithm implementation
- **DFACanvas:** Renders DFA with smooth arrows
- **App:** Main window, UI coordination

Data flows from grammar input → parsing → DFA construction → table generation
→ visualization

Executive Summary

This project implements a complete LR(0) parser generator with an interactive graphical interface. The system takes a context-free grammar as input and automatically constructs the LR(0) automaton (DFA), generates ACTION and GOTO parsing tables, and visualizes the result with clear state transitions. Built with Python and Tkinter.

Key features include grammar input with alternative productions, epsilon support, conflict detection, zoomable/panable DFA visualization, and color-coded parsing tables.

User Interface Design

Layout: Two-panel design with grammar input on left, DFA and table tabs on right.

Grammar Input Panel:

- Scrolled text area for grammar entry
- Example buttons (Arithmetic, Nested Parens, Balanced, Simple)
- Build button triggers parser construction
- Status bar shows results/errors

DFA Visualization Tab:

- Canvas with zoom (+/- buttons or mouse wheel)
- Pan support (click and drag)
- Fit button to center all states
- State boxes: blue header with state number, items with dot notation
- Red border for accept states
- Colored arrows with labels

Parsing Table Tab:

- Color-coded grid (blue for ACTION, gray for GOTO)
- Shift (sX) in light blue
- Reduce (rX) in light orange
- Accept (acc) in light green
- Conflict warnings displayed prominently

Color Scheme: Professional blue/gray palette with intuitive action colors.

Testing and Validation

Test Grammars:

Grammar	States	Productions	Conflicts	LR(0)?
Arithmetic	12	7	Yes	No
Balanced Parens	4	3	No	Yes
Nested Parens	8	5	No	Yes
Simple (aAb)	5	4	No	Yes

Output

Grammar Input

Grammar Input
One production per line. Use | for alternatives.
Use ϵ / eps for empty. Arrow: $\rightarrow$ or $\Rightarrow$

```
S  $\rightarrow$  A
A  $\rightarrow$  ( A B )
A  $\rightarrow$  ( )
B  $\rightarrow$  ( A )
B  $\rightarrow$  ( )
```

Examples:

Arithmetic

Nested Parens

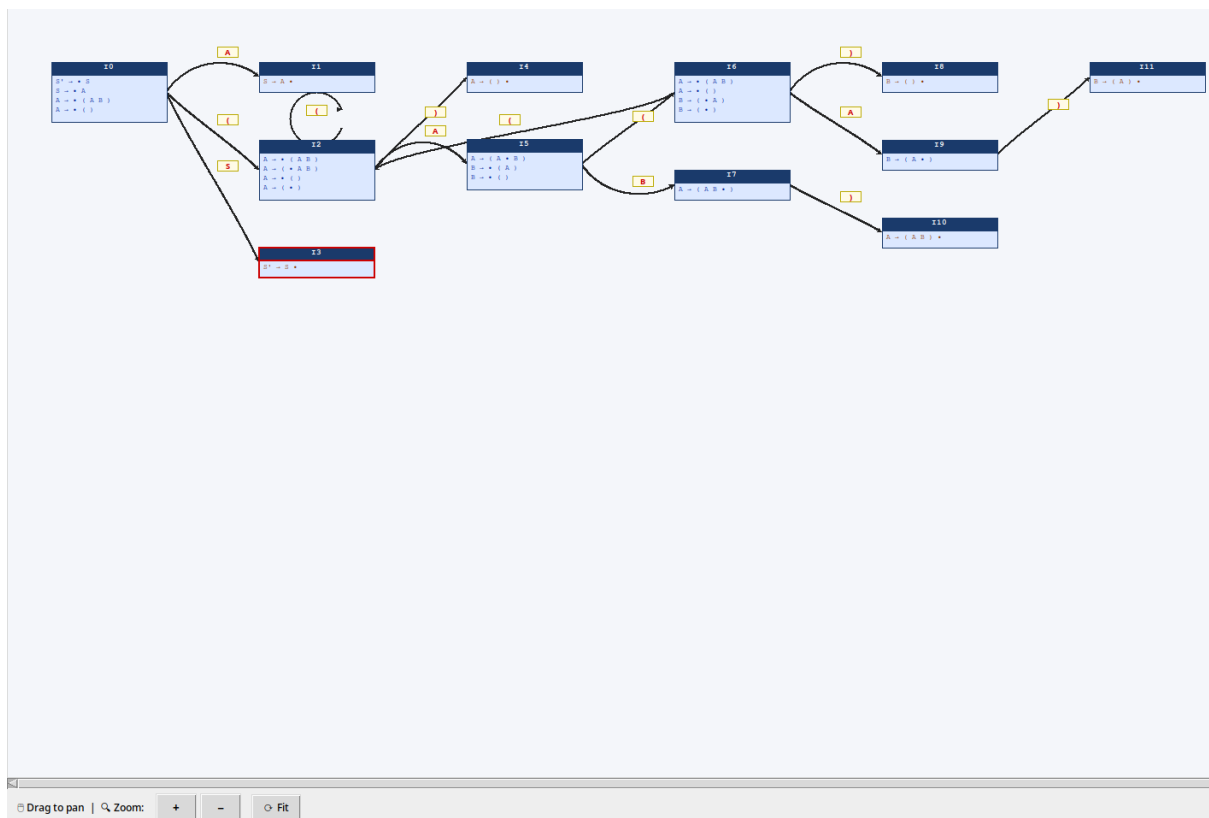
Balanced Parens

Simple (aAb)

► BUILD PARSER ◀

✔ Built: 12 states, 6 productions.

DFA



Parse Table

✓ No conflicts — grammar is LR(0).

State	\$	(	)	A	S	B
	ACTION	ACTION	ACTION	GOTO	GOTO	GOTO
0		s2		1	3	
1	r1	r1	r1			
2		s2	s4	5		
3	acc					
4	r3	r3	r3			
5		s6				7
6		s2	s8	9		
7			s10			
8	r5	r5	r5			
9			s11			
10	r2	r2	r2			
11	r4	r4	r4			

Conclusion

The LR(0) parser generator successfully implements all core algorithms (closure, goto, DFA construction, table generation) with a professional, interactive visualization. It serves as both a useful tool and an educational resource for understanding LR parsing fundamentals. The clear visualization helps bridge the gap between theoretical concepts and practical implementation.

Reference

Youtube: <https://youtu.be/J4ZME5KOB-s?si=9SrlnsR1poL1X8Q5>

Github: <https://github.com/Wajahat725/LR0-Parser.git>